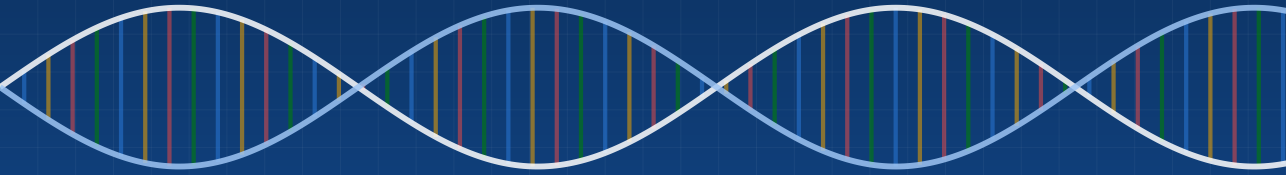


Alineamiento de secuencias



Nada tiene sentido en biología si no es a la luz de la evolución.

— Dobzhansky (1973)

● Al terminar este capítulo usted podrá

- Leer un *dot plot* e identificar en él identidad, repeticiones, inversiones e inserciones.
- Formular el alineamiento global y local como problemas de optimización y resolverlos por programación dinámica (Needleman-Wunsch y Smith-Waterman).
- Explicar de dónde salen las matrices PAM y BLOSUM como razones de verosimilitud logarítmicas, y justificar las penalizaciones afines de *gaps*.
- Interpretar un resultado de BLAST: puntuación en bits, valor *E* y los supuestos de la estadística de Karlin-Altschul.

Comparar es la operación más antigua de la biología. Linneo comparó flores; Darwin comparó picos de pinzones; nosotros comparamos cadenas de letras. Cuando dos secuencias se parecen más de lo que el azar permitiría, la explicación más sencilla es que comparten un ancestro, y de esa inferencia (la *homología*) depende casi todo lo que haremos en el resto del libro: anotar genes, predecir funciones, reconstruir

árboles, mapear lecturas o llamar variantes. Este capítulo construye, desde cero, la herramienta que hace posible esa comparación: el *alineamiento*.

Avanzaremos en cuatro pasos. Primero miraremos las secuencias con los ojos, mediante una matriz de puntos (sección 3.1). Luego convertiremos esa intuición visual en un algoritmo exacto, la programación dinámica (sección 3.2). Después nos preguntaremos cuánto debe valer cada coincidencia, cada sustitución y cada hueco, lo que nos llevará a las matrices de sustitución (sección 3.3). Por último, escalaremos el problema a bases de datos con millones de secuencias y aprenderemos a distinguir un parecido significativo de uno casual con BLAST y la estadística de Karlin-Altschul (sección 3.4).

3.1 Dot plots: mirar antes de calcular

 Colab · 3.1 · Dot plots

 repositorio del curso

Dos manuscritos sobre la mesa

Imagine que recibe dos copias manuscritas de un mismo poema, copiadas por escribas distintos con siglos de diferencia. Para compararlas, podría escribir una copia a lo largo del borde superior de una hoja cuadrículada y la otra a lo largo del borde izquierdo, y marcar con un punto cada casilla donde las dos letras coinciden. Allí donde los escribas copiaron fielmente, aparecerá una línea diagonal limpia; donde uno de ellos saltó un verso, la diagonal se desplazará; donde repitió una estrofa, aparecerá una segunda diagonal paralela. Sin hacer una sola cuenta, la hoja le habrá contado la historia de las copias.

Esa hoja cuadrículada es exactamente un *dot plot* (matriz de puntos), la representación propuesta por Gibbs y McIntyre (1970) cuando las secuencias de proteínas empezaban a acumularse y aún no existían algoritmos para compararlas. Su gran virtud es que no impone ninguna hipótesis: muestra *todas* las coincidencias a la vez, y deja al ojo humano, un detector de patrones extraordinario, la tarea de interpretarlas.

3.1.1 Definición formal

Sean $x = x_1 x_2 \dots x_n$ e $y = y_1 y_2 \dots y_m$ dos secuencias sobre un alfabeto $\mathcal{A}$ (por ejemplo, $\mathcal{A} = \{A, C, G, T\}$).

Definición 3.1. (Dot plot con ventana)

Dada una ventana de longitud impar w y un umbral $t \leq w$, el *dot plot* de x contra y es la matriz binaria $D \in \{0, 1\}^{n \times m}$ con

$$D_{ij} = \mathbb{1} \left[\sum_{k=-h}^h \mathbb{1}[x_{i+k} = y_{j+k}] \geq t \right], \quad h = \frac{w-1}{2}. \quad (3.1)$$

DONDE

D_{ij}	Celda de la fila i (posición en x) y la columna j (posición en y); vale 1 si se dibuja un punto.
$\mathbb{1}[\cdot]$	Función indicadora: 1 si la condición es verdadera, o si no.
w, h	Longitud de la ventana deslizante y su semiancho.
t	Número mínimo de coincidencias dentro de la ventana para marcar la celda.

Con $w = 1$ y $t = 1$ recuperamos la versión más ingenua: un punto por cada par de letras idénticas. En ADN esto es casi inútil, porque con cuatro letras la probabilidad de que dos posiciones al azar coincidan es $1/4$: una cuarta parte de la matriz se llenaría de ruido. La ventana resuelve el problema. Si las bases fueran independientes y equiprobables, el número de coincidencias en una ventana seguiría una distribución binomial $\text{Bin}(w, 1/4)$, y la probabilidad de un punto espurio sería

$$p_{\text{ruido}} = \sum_{k=t}^w \binom{w}{k} \left(\frac{1}{4}\right)^k \left(\frac{3}{4}\right)^{w-k}. \quad (3.2)$$

Para $w = 11$, $t = 7$, esta probabilidad es de apenas 0,0076: menos de un punto espurio por cada cien celdas. Esta es la idea de filtrado que Maizel y Lenk (1981) popularizaron y que, como veremos, reaparece en el corazón de BLAST.

3.1.2 Cómo se lee un dot plot

La figura 3.1 muestra un ejemplo construido a propósito: la secuencia y es una copia de x en la que un segmento central fue invertido y complementado. Aparecen los motivos que conviene aprender a reconocer:

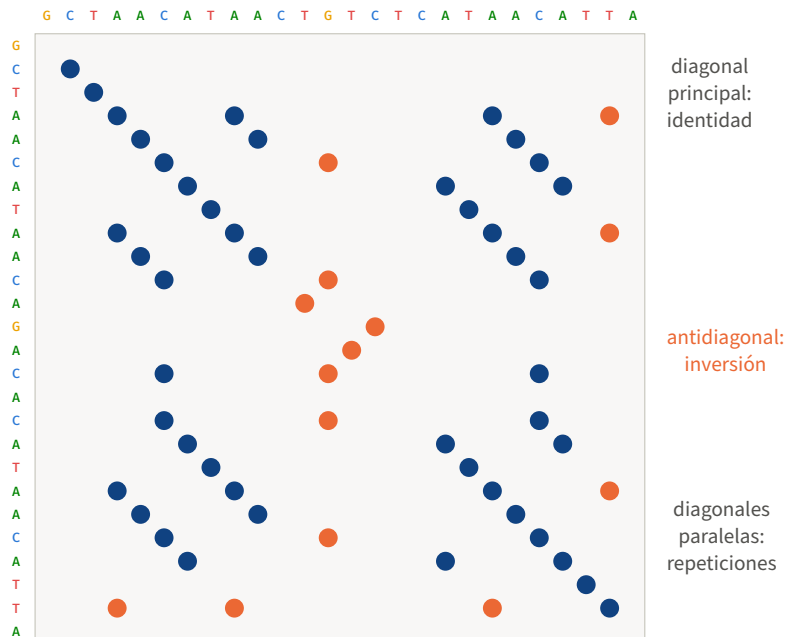


Figura 3.1. Anatomía de un *dot plot* ($w = 3, t = 3$). Las filas corresponden a x y las columnas a y . En azul, coincidencias directas: la diagonal principal y dos diagonales paralelas producidas por un segmento repetido. En naranja, coincidencias con el reverso complementario, que revelan el segmento invertido como una antidiagonal. Figura generada con `figuras/cap03/generar.py`.

- **Diagonal principal:** regiones idénticas u homólogas en el mismo orden.
- **Diagonales paralelas desplazadas:** repeticiones, pues un mismo segmento de x se parece a dos lugares de y .
- **Salto lateral de la diagonal:** inserciones o deleciones (*indels*); la diagonal continúa, pero desplazada una fila o una columna.
- **Antidiagonales:** inversiones; se detectan comparando x con el *reverso complementario* de y (puntos naranjas).
- **Bloques cuadrados:** regiones de baja complejidad (p. ej., **ATATATAT**), que coinciden consigo mismas en cualquier desfase.

Ejemplo 3.1. (Un dot plot en diez líneas)

El siguiente fragmento calcula la matriz de la ecuación (3.1) de forma vectorizada: en lugar de recorrer ventanas, se suman diagonales desplazadas de la matriz de identidad.

```
dotplot.py

import numpy as np

def dotplot(x: str, y: str, w: int = 11, t: int = 7) -> np.ndarray:
    """Matriz binaria D (n x m) con ventana w y umbral t."""
    X = np.frombuffer(x.encode(), dtype=np.uint8)
    Y = np.frombuffer(y.encode(), dtype=np.uint8)
    eq = (X[:, None] == Y[None, :]).astype(np.int16) # identidad
    ↪ letra a letra
    h = w // 2
    acc = np.zeros_like(eq)
    for k in range(-h, h + 1): # suma a lo
        ↪ largo de la diagonal
        acc += np.roll(np.roll(eq, k, axis=0), k, axis=1)
    return acc >= t
```

Con dos genomas de 30 kb la matriz tiene 9×10^8 celdas; por eso las herramientas reales, como `dotter` o `minimap2 -X`, calculan solo semillas de *k*-mers exactos en lugar de la matriz completa.

⚠ Lo que un dot plot no hace

Un *dot plot* no produce un alineamiento ni una puntuación. Es una herramienta de *exploración*: sugiere hipótesis (“aquí hay una duplicación en tándem”) que luego deben cuantificarse. Tampoco escala: su coste en memoria es $O(nm)$.

3.2 Programación dinámica: Needleman-Wunsch y Smith-Waterman

3.2.1 ¿Qué es un alineamiento?

Un *alineamiento* de x e y es una forma de escribir una secuencia encima de la otra, insertando huecos (*gaps*, representados con “-”) para que las posiciones que creemos que descienden de un mismo ancestro queden en la misma columna. Cada columna es de uno de tres tipos:

Columna	Ejemplo	Interpretación evolutiva
Coincidencia (<i>match</i>)	$\begin{pmatrix} A \\ A \end{pmatrix}$	la posición se conservó
Sustitución (<i>mismatch</i>)	$\begin{pmatrix} A \\ G \end{pmatrix}$	hubo una mutación puntual
Hueco (<i>gap</i>)	$\begin{pmatrix} A \\ - \end{pmatrix}, \begin{pmatrix} - \\ A \end{pmatrix}$	inserción en una rama o delección en la otra

Un alineamiento es, por lo tanto, una *hipótesis evolutiva* escrita en forma de tabla. Y como toda hipótesis, puede ser mejor o peor. Para compararlas asignamos a cada columna una puntuación: $s(a, b)$ para un par de letras y $-d$ por cada posición con hueco. La puntuación del alineamiento es la suma de sus columnas, y el *alineamiento óptimo* es el de puntuación máxima.

El problema de contar caminos

¿Por qué no probar todos los alineamientos y quedarnos con el mejor? Porque son astronómicamente muchos. El número de alineamientos globales de dos secuencias de longitud n crece aproximadamente como

$$\binom{2n}{n} \approx \frac{2^{2n}}{\sqrt{\pi n}},$$

de modo que para dos proteínas modestas de 300 residuos hay unos 10^{179} alineamientos, muchísimos más que átomos en el universo observable (unos 10^{80}). Necesitamos una idea más astuta que la fuerza bruta.

3.2.2 El principio de optimalidad

La idea astuta proviene de la investigación de operaciones. Bellman (1954) observó que muchos problemas de decisión secuencial satisfacen un *principio de optimalidad*: la mejor solución global contiene, como piezas, las mejores soluciones de subproblemas más pequeños. En el alineamiento, esto se traduce en una observación casi evidente una vez que se enuncia: la última columna de un alineamiento

óptimo solo puede ser de tres tipos, y lo que queda a su izquierda debe ser, a su vez, un alineamiento óptimo de los prefijos correspondientes. Si no lo fuera, podríamos sustituirlo por uno mejor y mejorar el total, lo que contradice la optimalidad. Eddy (2004a) ofrece una introducción breve y muy clara a esta forma de pensar.

3.2.3 Needleman-Wunsch: alineamiento global

Sea $F(i, j)$ la puntuación del mejor alineamiento entre los prefijos $x_{1..i}$ e $y_{1..j}$. Needleman y Wunsch (1970) mostraron que F se puede llenar como una tabla, fila por fila, con la recurrencia

Teorema 3.1. (Recurrencia de Needleman-Wunsch)

Con penalización lineal $d > 0$ por posición de hueco,

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + s(x_i, y_j) & (\text{alinear } x_i \text{ con } y_j) \\ F(i-1, j) - d & (x_i \text{ frente a un hueco}) \\ F(i, j-1) - d & (y_j \text{ frente a un hueco}) \end{cases} \quad (3.3)$$

con condiciones de frontera $F(0, 0) = 0$, $F(i, 0) = -id$ y $F(0, j) = -jd$. La puntuación óptima es $F(n, m)$.

DONDE

$F(i, j)$	Mejor puntuación posible alineando los primeros i símbolos de x con los primeros j de y .
$s(a, b)$	Puntuación de sustitución de la letra a por la b (positiva si se parecen).
d	Costo de cada posición alineada contra un hueco.
n, m	Longitudes de x e y .

La tabla se llena en $O(nm)$ operaciones, ya que cada celda consulta tres vecinas. Para recuperar el alineamiento, y no solo su puntuación, se recorre la tabla en sentido inverso (*traceback*) desde (n, m) , preguntando en cada celda cuál de los tres casos produjo el máximo. Una diagonal equivale a una columna de pares; un paso vertical, a una letra de x frente a un hueco; uno horizontal, a una letra de y frente a un hueco.

	ϵ	G	T	T	A	G	C	A
ϵ	0	-2	-4	-6	-8	-10	-12	-14
G	-2	1	-1	-3	-5	-7	-9	-11
A	-4	-1	0	-2	-2	-4	-6	-8
T	-6	-3	0	1	-1	-3	-5	-7
T	-8	-5	-2	1	0	-2	-4	-6
A	-10	-7	-4	-1	2	0	-2	-3
C	-12	-9	-6	-3	0	1	1	-1
A	-14	-11	-8	-5	-2	-1	0	2

Figura 3.2. Matriz de programación dinámica para $x = \text{GATTACA}$ (filas) e $y = \text{GTTAGCA}$ (columnas), con $s = +1$ para coincidencias, -1 para sustituciones y $d = 2$. Las celdas resaltadas y las flechas naranjas marcan el *traceback* desde la esquina inferior derecha. Los valores negativos aparecen en rojo.

Ejemplo 3.2. (Leyendo la matriz de la figura 3.2)

La puntuación óptima es $F(7, 7) = 2$. Siguiendo las flechas desde la esquina se obtiene

G	A	T	T	A	-	C	A
G	-	T	T	A	G	C	A

Seis coincidencias (+6) y dos huecos (-4) dan $6 - 4 = 2$, lo que confirma el valor de la esquina. Observe el primer paso vertical tras la G: el algoritmo “decidió” que era mejor pagar un hueco que forzar una A contra una T, porque el hueco permite que el bloque TTA quede perfectamente alineado.

needleman_wunsch.py

```
def needleman_wunsch(x, y, match=1, mismatch=-1, gap=2):
    n, m = len(x), len(y)
```

```

F = [[0] * (m + 1) for _ in range(n + 1)]
for i in range(1, n + 1): F[i][0] = -i * gap          # frontera: solo
↳ huecos
for j in range(1, m + 1): F[0][j] = -j * gap
for i in range(1, n + 1):
    for j in range(1, m + 1):
        s = match if x[i - 1] == y[j - 1] else mismatch
        F[i][j] = max(F[i - 1][j - 1] + s,          # diagonal
                      F[i - 1][j] - gap,            # hueco en y
                      F[i][j - 1] - gap)           # hueco en x
return F[n][m], F

```

3.2.4 Smith-Waterman: alineamiento local

El alineamiento global asume que las dos secuencias son homólogas de extremo a extremo. Muchas veces no es así: dos proteínas pueden compartir un único dominio en medio de regiones sin relación, o una lectura corta de secuenciación puede provenir de un pequeño trozo de un cromosoma. En esos casos queremos el *par de subcadenas* más parecido. Smith y Waterman (1981) lo consiguieron con un cambio mínimo pero profundo: permitir que un alineamiento “vuelva a empezar” en cualquier celda.

Teorema 3.2. (Recurrencia de Smith-Waterman)

$$H(i, j) = \max \begin{cases} 0 & (\text{empezar de nuevo}) \\ H(i - 1, j - 1) + s(x_i, y_j) \\ H(i - 1, j) - d \\ H(i, j - 1) - d \end{cases} \quad (3.4)$$

con $H(i, 0) = H(0, j) = 0$. La puntuación local óptima es $\max_{i,j} H(i, j)$ y el traceback termina al encontrar un 0.

El cero cumple un papel decisivo. Un prefijo que acumula puntuación negativa nunca ayuda a una extensión posterior, así que la recurrencia lo “olvida”. Para que esto funcione, la puntuación esperada de un par aleatorio debe ser negativa, $\sum_{a,b} p_a p_b s(a, b) < 0$; de lo contrario, los alineamientos locales crecerían indefinidamente y se volverían, en la práctica, globales. Esta condición reaparecerá como supuesto central de la estadística de BLAST.

🔗 Sellers, Waterman y la métrica evolutiva

Entre Needleman-Wunsch (1970) y Smith-Waterman (1981) media una década de formalización matemática. Sellers (1974) demostró que, bajo ciertas condiciones, la distancia de edición entre secuencias es una métrica genuina, lo que dio al alineamiento un fundamento geométrico. Temple Smith y Michael Waterman, un físico y un matemático, trabajaban en Los Álamos cuando publicaron su algoritmo en un artículo de apenas tres páginas, hoy uno de los más citados de la biología computacional.

3.2.5 Huecos afines y el algoritmo de Gotoh

Con una penalización lineal, un hueco de 10 posiciones cuesta lo mismo que diez huecos de 1 posición dispersos. Biológicamente esto es absurdo: un único evento de inserción o delección suele afectar a varias bases contiguas, y abrir un hueco es mucho más raro que alargarlo. La solución estándar es la *penalización afín*:

$$\gamma(g) = -(d_o + (g - 1) d_e), \quad d_o > d_e > 0, \quad (3.5)$$

DONDE

g	Longitud del hueco (número de posiciones consecutivas).
d_o	Costo de <i>abrir</i> el hueco (primer símbolo).
d_e	Costo de <i>extender</i> el hueco en cada posición adicional.

Una función afín rompe la recurrencia simple, porque el costo de una celda depende de si el hueco ya estaba abierto. Gotoh (1982) resolvió el problema usando tres matrices acopladas, una por cada “estado” en el que puede terminar un alineamiento:

$$M(i, j) = s(x_i, y_j) + \max\{M, I_x, I_y\}(i-1, j-1), \quad (3.6)$$

$$I_x(i, j) = \max\{M(i-1, j) - d_o, I_x(i-1, j) - d_e\}, \quad (3.7)$$

$$I_y(i, j) = \max\{M(i, j-1) - d_o, I_y(i, j-1) - d_e\}. \quad (3.8)$$

El coste sigue siendo $O(nm)$ en tiempo. Esta arquitectura de tres estados no es un truco aislado: es exactamente un *modelo oculto de Márkov de pares*, como mostraron Durbin et al. (1998), y la retomaremos en el ?? al estudiar perfiles y HMMER.

Memoria lineal: el truco de Hirschberg

Para alinear dos cromosomas bacterianos de 5 Mb, una matriz $n \times m$ necesitaría $2,5 \times 10^{13}$ celdas. Hirschberg (1975) mostró que el alineamiento óptimo puede recuperarse en espacio $O(n + m)$: se calcula la fila central de la matriz hacia adelante y hacia atrás, se localiza la celda por la que pasa el camino óptimo y se resuelven recursivamente las dos mitades. El tiempo apenas se duplica, pero la memoria se reduce en órdenes de magnitud.

3.3 Matrices de sustitución y penalización de gaps



Colab · 3.3 · Matrices de sustitución



repositorio del curso

Hasta ahora hemos usado puntuaciones arbitrarias: +1 para coincidencias, -1 para sustituciones. En ADN esa simplificación es tolerable; en proteínas, no. Cambiar una leucina por una isoleucina (ambas hidrofóbicas, casi del mismo tamaño) apenas altera una proteína, mientras que cambiar un triptófano por una glicina puede destruir su plegamiento. Necesitamos puntuaciones que reflejen la química y la evolución.

3.3.1 Puntuaciones como razones de verosimilitud

La pregunta correcta es estadística. Frente a un par alineado (a, b) , comparamos dos hipótesis:

- $\mathcal{H}_1$ (*homología*): a y b descienden de un ancestro común y aparecen juntos con probabilidad q_{ab} , estimada a partir de alineamientos confiables.
- $\mathcal{H}_0$ (*azar*): a y b se eligieron de forma independiente, con probabilidad $p_a p_b$ según sus frecuencias de fondo.

La puntuación natural es el logaritmo de la razón de verosimilitudes:

$$s(a, b) = \frac{1}{\lambda} \log \frac{q_{ab}}{p_a p_b}. \quad (3.9)$$

DONDE

q_{ab}	Frecuencia con que a y b aparecen alineados en secuencias homólogas.
p_a, p_b	Frecuencias de fondo de cada aminoácido en las proteínas.
λ	Factor de escala; con $\lambda = \ln 2/2$ las puntuaciones quedan en medios bits.

Esta forma tiene una consecuencia elegante: como el logaritmo convierte productos en sumas, sumar las puntuaciones de las columnas equivale a multiplicar probabilidades, siempre que las columnas se consideren independientes. La puntuación de un alineamiento completo es, por lo tanto, el logaritmo de la razón de verosimilitudes de la hipótesis de homología frente a la de azar. Altschul (1991) demostró además que *cualquier* matriz de puntuación con esperanza negativa y al menos un valor positivo es implícitamente una matriz de este tipo, para algún conjunto de frecuencias q_{ab} “objetivo”.

3.3.2 PAM: extrapolar la evolución

Margaret Dayhoff y sus colaboradores construyeron las primeras matrices a partir de 71 familias de proteínas muy similares, con al menos 85 % de identidad (Dayhoff et al., 1978). Contaron las sustituciones observadas en árboles filogenéticos y las normalizaron para obtener una matriz de transición $P^{(1)}$ que representa 1 PAM: la divergencia en la que, en promedio, ha cambiado un 1 % de los aminoácidos (*point accepted mutation*). Suponiendo que la evolución es un proceso de Márkov, la matriz para distancias mayores se obtiene elevando a potencias:

$$P^{(k)} = \left(P^{(1)}\right)^k, \quad s_k(a, b) = \frac{1}{\lambda} \log \frac{p_a P_{ab}^{(k)}}{p_a p_b} = \frac{1}{\lambda} \log \frac{P_{ab}^{(k)}}{p_b}. \quad (3.10)$$

La PAM250, muy usada en su época, corresponde a una distancia en la que sobreviven solo alrededor del 20 % de las identidades. El punto débil del método es precisamente la extrapolación: los errores de estimación de $P^{(1)}$ se amplifican al elevarla a la potencia 250.

3.3.3 BLOSUM: observar directamente

Henikoff y Henikoff (1992) tomaron el camino opuesto. En lugar de extrapolar desde secuencias cercanas, usaron la base de datos BLOCKS: unos 2000 segmentos conservados, sin huecos, de más de 500 familias de proteínas. Para construir la

	C	S	T	A	G	P	D	E	Q	N	H	R	K	M	I	L	V	W	Y	F	
C	9	-1	-1	0	-3	-3	-3	-4	-3	-3	-3	-3	-3	-1	-1	-1	-1	-2	-2	-2	
S	-1	4	1	1	0	-1	0	0	0	1	-1	-1	0	-1	-2	-2	-2	-3	-2	-2	
T	-1	1	5	0	-2	-1	-1	-1	-1	0	-2	-1	-1	-1	-1	-1	0	-2	-2	-2	pequeños
A	0	1	0	4	0	-1	-2	-1	-1	-2	-2	-1	-1	-1	-1	-1	0	-3	-2	-2	
G	-3	0	-2	0	6	-2	-1	-2	-2	0	-2	-2	-2	-3	-4	-4	-3	-2	-3	-3	
P	-3	-1	-1	-1	-2	7	-1	-1	-1	-2	-2	-2	-1	-2	-3	-3	-2	-4	-3	-4	
D	-3	0	-1	-2	-1	-1	6	2	0	1	-1	-2	-1	-3	-3	-4	-3	-4	-3	-3	ácidos/amidas
E	-4	0	-1	-1	-2	-1	2	5	2	0	0	0	1	-2	-3	-3	-2	-3	-2	-3	
Q	-3	0	-1	-1	-2	-1	0	2	5	0	0	1	1	0	-3	-2	-2	-2	-1	-3	
N	-3	1	0	-2	0	-2	1	0	0	6	1	0	0	0	-2	-3	-3	-3	-4	-2	-3
H	-3	-1	-2	-2	-2	-2	-1	0	0	1	8	0	-1	-2	-3	-3	-3	-2	2	-1	básicos
R	-3	-1	-1	-1	-2	-2	-2	0	1	0	0	5	2	-1	-3	-2	-3	-3	-2	-3	
K	-3	0	-1	-1	-2	-1	-1	1	1	0	-1	2	5	-1	-3	-2	-2	-3	-2	-3	
M	-1	-1	-1	-1	-3	-2	-3	-2	0	-2	-2	-1	-1	5	1	2	1	-1	-1	0	alifáticos
I	-1	-2	-1	-1	-4	-3	-3	-3	-3	-3	-3	-3	-3	1	4	2	3	-3	-1	0	
L	-1	-2	-1	-1	-4	-3	-4	-3	-2	-3	-3	-2	-2	2	2	4	1	-2	-1	0	
V	-1	-2	0	0	-3	-2	-3	-2	-2	-3	-3	-3	-2	1	3	1	4	-3	-1	-1	
W	-2	-3	-2	-3	-2	-4	-4	-3	-2	-4	-2	-3	-3	-1	-3	-2	-3	11	2	1	aromáticos
Y	-2	-2	-2	-2	-3	-3	-3	-2	-1	-2	2	-2	-2	-1	-1	-1	-1	2	7	3	
F	-2	-2	-2	-2	-3	-4	-3	-3	-3	-3	-1	-3	-3	0	0	0	-1	1	3	6	

Figura 3.3. La matriz BLOSUM62, en medios bits, con los aminoácidos ordenados por propiedades fisicoquímicas. Los bloques recuadrados en la diagonal muestran que las sustituciones *dentro* de un grupo (p. ej., $I \leftrightarrow L \leftrightarrow V$) reciben puntuaciones positivas o cercanas a cero, mientras que las sustituciones entre grupos son penalizadas. El valor de W–W (+11) refleja que el triptófano es raro y muy conservado. Valores tomados de Biopython (Cock et al., 2009).

BLOSUM L agruparon las secuencias de cada bloque con más de L % de identidad, con el fin de que las familias muy representadas no dominaran los conteos, y estimaron q_{ab} directamente de los pares observados. Así, la BLOSUM62 se basa en secuencias con a lo sumo 62 % de identidad, y la BLOSUM45 en otras aún más divergentes.

Número alto en PAM $\Rightarrow$ secuencias más divergentes.
 Número alto en BLOSUM $\Rightarrow$ secuencias más cercanas.

Un error afortunado

Una década y media después de su publicación, Styczynski et al. (2008) descubrieron que el programa original que generó las BLOSUM contenía un error en el agrupamiento de secuencias. Paradójicamente, la matriz “incorrecta” encuentra homólogos remotos *mejor* que la versión corregida, y por eso la BLOSUM62 original sigue siendo la matriz por defecto de BLAST. Eddy (2004b) explica, en dos páginas, cómo reconstruirla desde los datos.

3.3.4 Elegir la matriz y los huecos

No existe una matriz universalmente óptima. Cada una está calibrada para un nivel de divergencia: una matriz “profunda” (BLOSUM45, PAM250) detecta homólogos lejanos pero puntúa en exceso los parecidos casuales en alineamientos cortos, mientras que una matriz “superficial” (BLOSUM80, PAM30) hace lo contrario. En la práctica, BLOSUM62 con apertura $d_o = 11$ y extensión $d_e = 1$ es el punto de partida de `blastp`; para péptidos cortos se recomienda PAM30 (Pearson, 2013).

Parámetros que no se pueden mezclar

Las penalizaciones de hueco no son independientes de la matriz: ambas deben estar en la misma escala y, para búsquedas en bases de datos, calibradas conjuntamente, porque de ellas dependen los parámetros λ y K de la sección siguiente. Cambiar d_o sin recalibrar invalida los valores E .

3.4 BLAST y la estadística de Karlin-Altschul

 Colab · 3.4 · BLAST y valores E

 repositorio del curso

3.4.1 El problema de escala

Smith-Waterman es exacto, pero comparar una proteína de 300 residuos contra UniProt (unos 10^{11} residuos) requeriría 3×10^{13} celdas por consulta. A finales de los ochenta, FASTA (Pearson & Lipman, 1988) mostró que se podía ganar velocidad buscando primero palabras cortas idénticas. BLAST (Altschul et al., 1990) llevó esa idea más lejos con una estrategia de tres pasos, conocida como *seed-and-extend*:

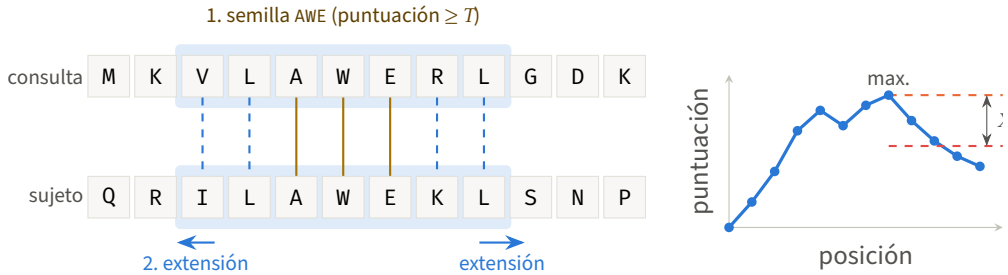


Figura 3.4. La estrategia *seed-and-extend*. **Izquierda:** una palabra de la consulta (AWE) encuentra una semilla en el sujeto; la extensión sin huecos añade pares vecinos mientras la puntuación acumulada lo justifique. **Derecha:** perfil de puntuación durante la extensión; se abandona cuando la puntuación cae X unidades por debajo del máximo, y el HSP se recorta en ese máximo.

- 1. Siembra.** Se divide la consulta en palabras de longitud w (3 para proteínas, 11 o más para ADN) y, para cada una, se genera la *vecindad* de palabras que puntúan al menos T contra ella con la matriz de sustitución.
- 2. Búsqueda.** Un índice precalculado de la base de datos localiza en tiempo casi constante todas las apariciones de esas palabras (*hits*).
- 3. Extensión.** Cada *hit* se extiende en ambas direcciones y se detiene cuando la puntuación cae más de X unidades por debajo del máximo alcanzado (*X-drop*). Los segmentos resultantes son los *high-scoring segment pairs* (HSP).

La versión de 1997 (Altschul et al., 1997) añadió dos mejoras decisivas: la heurística de “dos *hits*”, que exige dos semillas cercanas en la misma diagonal antes de extender y reduce el trabajo a la mitad, y la extensión *con huecos* mediante programación dinámica restringida a una banda. La implementación moderna, BLAST+ (Camacho et al., 2009), reorganizó el código en C++ y es la que usaremos en el laboratorio.

3.4.2 ¿Cuándo un parecido es significativo?

Toda búsqueda produce un mejor resultado, incluso si la consulta es una secuencia aleatoria. La pregunta crucial es: *¿qué puntuación esperaríamos por puro azar?* La respuesta, obtenida por Karlin y Altschul (1990), es uno de los resultados más bellos de la bioinformática.

Consideremos la puntuación máxima S de un alineamiento local *sin huecos* entre dos secuencias aleatorias de longitudes m y n . La suma de puntuaciones a lo largo de una diagonal es un paseo aleatorio con deriva negativa (porque $E[s] < 0$). Los

segmentos de alta puntuación son *excursiones* excepcionales de ese paseo, y la teoría de valores extremos predice que su máximo sigue una distribución de Gumbel.

Teorema 3.3. (Karlin-Altschul)

Si $\sum_{a,b} p_a p_b s(a,b) < \infty$ y existe al menos un par con $s(a,b) > 0$, el número esperado de HSP distintos con puntuación $\geq S$ es, asintóticamente,

$$E = K m n e^{-\lambda S}, \quad (3.11)$$

donde $\lambda > 0$ es la única raíz positiva de

$$\sum_{a,b} p_a p_b e^{\lambda s(a,b)} = 1. \quad (3.12)$$

El número de HSP con puntuación $\geq S$ sigue una distribución de Poisson de media E , de modo que $\mathbb{P}(\text{al menos uno}) = 1 - e^{-E}$.

DONDE

E	Valor E (<i>expect value</i>): cuántos resultados tan buenos esperaríamos por azar en esta búsqueda.
m, n	Longitud de la consulta y tamaño total de la base de datos, en residuos.
S	Puntuación bruta del alineamiento, en las unidades de la matriz.
λ	Escala natural de la matriz; convierte puntuaciones en unidades de información.
K	Constante que corrige por la dependencia entre HSP solapados (típicamente $\approx 0,1$).

La ecuación (3.12) merece una pausa. Si comparamos con la ecuación (3.9), vemos que $e^{\lambda s(a,b)} = q_{ab}/(p_a p_b)$, de modo que la condición dice simplemente que las frecuencias objetivo q_{ab} suman 1. El mismo λ que define la matriz como razón de verosimilitudes gobierna la estadística de sus máximos. Teoría de la información y teoría de valores extremos se encuentran en una sola ecuación.

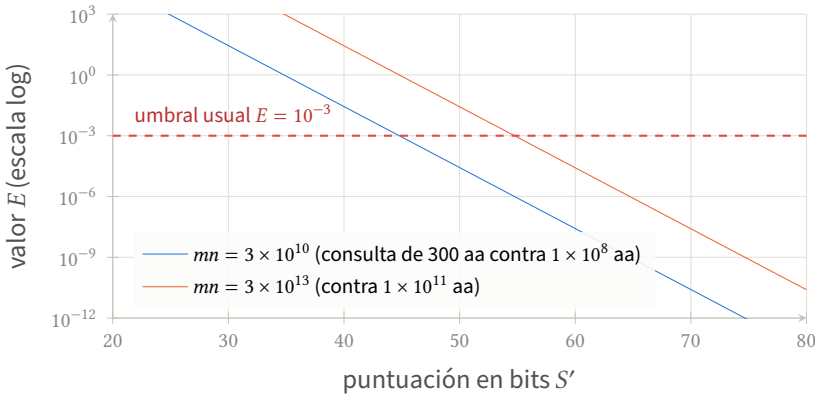


Figura 3.5. El valor E decrece exponencialmente con la puntuación en bits (ecuación (3.13)). La misma puntuación que es muy significativa contra una base de datos pequeña (azul) puede no serlo contra una mil veces mayor (naranja): el tamaño del espacio de búsqueda forma parte de la evidencia.

3.4.3 Puntuación en bits

Como λ y K dependen de la matriz, las puntuaciones brutas no son comparables entre búsquedas. BLAST las normaliza a *bits*:

$$S' = \frac{\lambda S - \ln K}{\ln 2}, \quad \text{de modo que} \quad E = mn 2^{-S'}. \quad (3.13)$$

Esta forma es muy práctica: cada bit adicional de puntuación reduce a la mitad el número esperado de falsos positivos, y para saber si un resultado es significativo basta comparar S' con $\log_2(mn)$, que es el número de bits necesarios para “destacar” en un espacio de búsqueda de tamaño mn .

Ejemplo 3.3. (Interpretar un resultado real)

Una búsqueda `blastp` de una proteína de 350 aminoácidos contra una base de datos de 2×10^{11} residuos devuelve un resultado con puntuación de 85 bits. El valor esperado es

$$E = 350 \times 2 \times 10^{11} \times 2^{-85} \approx 7 \times 10^{13} \times 2,6 \times 10^{-26} \approx 1,8 \times 10^{-12}.$$

Esperaríamos encontrar por azar un resultado así una vez en cada medio billón de búsquedas: la homología es muy probable. En cambio, un resultado de 40 bits en la misma búsqueda tendría $E \approx 64$: decenas de resultados tan

buenos aparecerían por azar, así que no debe interpretarse como evidencia de homología.

```
blast_local.py

import subprocess
from Bio import Blast                                # Biopython >=
↳ 1.83

subprocess.run(["blastp", "-query", "consulta.fasta", "-db", "swissprot",
               "-evalue", "1e-3", "-matrix", "BLOSUM62",
               "-gapopen", "11", "-gapextend", "1",
               "-outfmt", "5", "-out", "resultado.xml"], check=True)
with open("resultado.xml", "rb") as fh:
    for hit in Blast.read(fh):                        # un Hit por
↳ secuencia sujeto
        hsp = hit[0]                                  # mejor HSP
        print(f"{hit.target.description[:40]:40s} "
              f"{hsp.annotations['bit score']:6.1f} bits "
              f"E={hsp.annotations['evalue']:.1e}")
```

3.4.4 Los supuestos, y cuándo fallan

La elegancia de la ecuación (3.11) no debe ocultar sus supuestos. La teoría es exacta solo para alineamientos *sin huecos* entre secuencias *largas* de composición *aleatoria e independiente*. Para alineamientos con huecos no existe una fórmula cerrada: λ y K se estiman por simulación para cada combinación de matriz y penalizaciones, y las secuencias cortas requieren una corrección de “longitud efectiva” (Altschul et al., 1997; Karlin et al., 1990). Además, las regiones de baja complejidad y los sesgos de composición violan la independencia; por eso BLAST enmascara esas regiones (con SEG o DUST) y aplica ajustes de composición a la matriz.

⚠ Tres errores frecuentes al leer un valor E

1. *Tratarlo como una probabilidad.* E es un número esperado y puede ser mayor que 1. Solo cuando $E < 0,01$ se cumple que $E \approx p$.
2. *Compararlo entre bases de datos.* Crece linealmente con n : la misma búsqueda contra una base de datos diez veces mayor produce un E diez veces mayor.

3. *Confundir significancia con homología global.* Un E minúsculo indica que *algún segmento* es homólogo; puede tratarse de un único dominio compartido entre proteínas por lo demás distintas.

☰ En resumen

- Un *dot plot* muestra todas las similitudes entre dos secuencias; las ventanas con umbral filtran el ruido y permiten ver identidades, repeticiones, inversiones e *indels*.
- El principio de optimalidad de Bellman reduce la búsqueda del mejor alineamiento, entre $\sim 10^{179}$ posibles, a llenar una tabla en $O(nm)$. Needleman-Wunsch alinea globalmente; Smith-Waterman, localmente, gracias al cero de la recurrencia.
- Los huecos afines modelan *indels* de varias bases y requieren tres matrices (Gotoh), equivalentes a un HMM de pares.
- Las matrices PAM y BLOSUM son razones de verosimilitud logarítmicas: PAM extrapola desde secuencias cercanas; BLOSUM observa directamente bloques divergentes.
- BLAST acelera la búsqueda con semillas y extensiones; su valor $E = Kmn e^{-\lambda S}$ cuantifica cuántos resultados así esperaríamos por azar, y la puntuación en bits lo hace comparable entre búsquedas.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 3.1 ●●● Dibuje a mano el *dot plot* ($w = 1$) de **ACGTACGT** contra sí misma. ¿Cuántas diagonales aparecen y qué representa cada una?
- 3.2 ●●● Usando la ecuación (3.2), calcule la probabilidad de un punto espurio con $w = 15$, $t = 10$ para ADN y para proteínas (suponga 20 aminoácidos equiprobables). ¿Por qué los *dot plots* de proteínas pueden usar ventanas más cortas?

- 3.3** ●●● Complete a mano la matriz de Needleman-Wunsch para GAATTC contra GATTA con $+2/-1$ y $d = 2$. Encuentre *todos* los alineamientos óptimos. ¿Por qué puede haber más de uno?
- 3.4** ●●● Modifique la función `needleman_wunsch` para implementar Smith-Waterman, devolviendo la puntuación máxima y la posición donde se alcanza.
- 3.5** ●●● Demuestre que, si $E[s] > 0$, la puntuación de Smith-Waterman entre dos secuencias aleatorias crece linealmente con su longitud. ¿Qué implica esto para el uso de matrices de puntuación de identidad pura ($+1/0$)?
- 3.6** ●●● Para ADN con $s = +1$ (coincidencia) y $s = -1$ (sustitución), y bases equiprobables, resuelva la ecuación (3.12) para λ . Compruebe su resultado numéricamente con `scipy.optimize.brentq`.
- 3.7** ●●● Simule 10 000 pares de secuencias aleatorias de 200 aminoácidos, calcule la puntuación local máxima sin huecos con BLOSUM62 y ajuste una distribución de Gumbel. Compare el λ estimado con el teórico (0,318 en unidades naturales).

Referencias del capítulo

- Altschul, S. F. (1991). Amino acid substitution matrices from an information theoretic perspective. *Journal of Molecular Biology*, 219(3), 555-565. [https://doi.org/10.1016/0022-2836\(91\)90193-A](https://doi.org/10.1016/0022-2836(91)90193-A)
- Altschul, S. F., Gish, W., Miller, W., Myers, E. W., & Lipman, D. J. (1990). Basic local alignment search tool. *Journal of Molecular Biology*, 215(3), 403-410. [https://doi.org/10.1016/S0022-2836\(05\)80360-2](https://doi.org/10.1016/S0022-2836(05)80360-2)
- Altschul, S. F., Madden, T. L., Schäffer, A. A., Zhang, J., Zhang, Z., Miller, W., & Lipman, D. J. (1997). Gapped BLAST and PSI-BLAST: A new generation of protein database search programs. *Nucleic Acids Research*, 25(17), 3389-3402. <https://doi.org/10.1093/nar/25.17.3389>
- Bellman, R. (1954). The theory of dynamic programming. *Bulletin of the American Mathematical Society*, 60(6), 503-515. <https://doi.org/10.1090/S0002-9904-1954-09848-8>
- Camacho, C., Coulouris, G., Avagyan, V., Ma, N., Papadopoulos, J., Bealer, K., & Madden, T. L. (2009). BLAST+: Architecture and applications. *BMC Bioinformatics*, 10, 421. <https://doi.org/10.1186/1471-2105-10-421>

- Cock, P. J. A., Antao, T., Chang, J. T., Chapman, B. A., Cox, C. J., Dalke, A., Friedberg, I., Hamelryck, T., Kauff, F., Wilczynski, B., & de Hoon, M. J. L. (2009). Biopython: Freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422-1423. <https://doi.org/10.1093/bioinformatics/btp163>
- Dayhoff, M. O., Schwartz, R. M., & Orcutt, B. C. (1978). A model of evolutionary change in proteins. En M. O. Dayhoff (Ed.), *Atlas of protein sequence and structure* (pp. 345-352, Vol. 5). National Biomedical Research Foundation.
- Dobzhansky, T. (1973). Nothing in biology makes sense except in the light of evolution. *The American Biology Teacher*, 35(3), 125-129. <https://doi.org/10.2307/4444260>
- Durbin, R., Eddy, S. R., Krogh, A., & Mitchison, G. (1998). *Biological sequence analysis: Probabilistic models of proteins and nucleic acids*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511790492>
- Eddy, S. R. (2004a). What is dynamic programming? *Nature Biotechnology*, 22(7), 909-910. <https://doi.org/10.1038/nbt0704-909>
- Eddy, S. R. (2004b). Where did the BLOSUM62 alignment score matrix come from? *Nature Biotechnology*, 22(8), 1035-1036. <https://doi.org/10.1038/nbt0804-1035>
- Gibbs, A. J., & McIntyre, G. A. (1970). The diagram, a method for comparing sequences: Its use with amino acid and nucleotide sequences. *European Journal of Biochemistry*, 16(1), 1-11. <https://doi.org/10.1111/j.1432-1033.1970.tb01046.x>
- Gotoh, O. (1982). An improved algorithm for matching biological sequences. *Journal of Molecular Biology*, 162(3), 705-708. [https://doi.org/10.1016/0022-2836\(82\)90398-9](https://doi.org/10.1016/0022-2836(82)90398-9)
- Henikoff, S., & Henikoff, J. G. (1992). Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences*, 89(22), 10915-10919. <https://doi.org/10.1073/pnas.89.22.10915>
- Hirschberg, D. S. (1975). A linear space algorithm for computing maximal common subsequences. *Communications of the ACM*, 18(6), 341-343. <https://doi.org/10.1145/360825.360861>
- Karlin, S., & Altschul, S. F. (1990). Methods for assessing the statistical significance of molecular sequence features by using general scoring schemes. *Proceedings of the National Academy of Sciences*, 87(6), 2264-2268. <https://doi.org/10.1073/pnas.87.6.2264>
- Karlin, S., Dembo, A., & Kawabata, T. (1990). Statistical composition of high-scoring segments from molecular sequences. *The Annals of Statistics*, 18(2), 571-581. <https://doi.org/10.1214/aos/1176347616>
- Maizel, J. V., & Lenk, R. P. (1981). Enhanced graphic matrix analysis of nucleic acid and protein sequences. *Proceedings of the National Academy of Sciences*, 78(12), 7665-7669. <https://doi.org/10.1073/pnas.78.12.7665>
- Needleman, S. B., & Wunsch, C. D. (1970). A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48(3), 443-453. [https://doi.org/10.1016/0022-2836\(70\)90057-4](https://doi.org/10.1016/0022-2836(70)90057-4)

- Pearson, W. R. (2013). An introduction to sequence similarity (“homology”) searching. *Current Protocols in Bioinformatics*, 42(1), 3.1.1-3.1.8. <https://doi.org/10.1002/0471250953.bi0301s42>
- Pearson, W. R., & Lipman, D. J. (1988). Improved tools for biological sequence comparison. *Proceedings of the National Academy of Sciences*, 85(8), 2444-2448. <https://doi.org/10.1073/pnas.85.8.2444>
- Sellers, P. H. (1974). On the theory and computation of evolutionary distances. *SIAM Journal on Applied Mathematics*, 26(4), 787-793. <https://doi.org/10.1137/0126070>
- Smith, T. F., & Waterman, M. S. (1981). Identification of common molecular subsequences. *Journal of Molecular Biology*, 147(1), 195-197. [https://doi.org/10.1016/0022-2836\(81\)90087-5](https://doi.org/10.1016/0022-2836(81)90087-5)
- Styczynski, M. P., Jensen, K. L., Rigoutsos, I., & Stephanopoulos, G. (2008). BLOSUM62 miscalculations improve search performance. *Nature Biotechnology*, 26(3), 274-275. <https://doi.org/10.1038/nbt0308-274>